

Department of Computer Science and Engineering

University of Dhaka

3rd Year, 2nd Semester — Session: 2022–2023

Singleton Pattern with MongoDB

Experimental Report

CSE 3216: Software Design Patterns Lab

Submitted By

Farhan Bin Rabbani

Roll: 35

Session: 2022-23

Dept. of CSE, University of Dhaka

Submitted To

Dr. Sarker Tanveer Ahmed Rume

Associate Professor

Md. Tanvir Alam

Lecturer

Submission Date: 21 April, 2026

Contents

1	Introduction	2
2	Eager Initialization	2
2.1	Overview	2
2.2	Test Output	2
2.3	Results	2
3	Lazy Initialization	3
3.1	Overview	3
3.2	Test Output	3
3.3	Results	3
4	Synchronized Method	3
4.1	Overview	3
4.2	Test Output	4
4.3	Results	4
5	Double-Checked Locking	4
5.1	Overview	4
5.2	Test Output	5
5.3	Results	5
6	Bill Pugh Implementation	5
6.1	Overview	5
6.2	Test Output	6
6.3	Results	6
7	Enum-Based Singleton	6
7.1	Overview	6
7.2	Test Output	7
7.3	Results	7
8	Comparative Analysis	7
8.1	Instance Validation Comparison	7
8.2	Thread Safety Comparison	8
8.3	Performance Comparison	8
9	Summary	9
9.1	Key Findings	9

1 Introduction

This report presents the empirical results of six Singleton design pattern implementations integrated with a MongoDB connection manager. The experiment evaluates three key criteria for each implementation: instance consistency across multiple calls, thread safety under concurrent access using Python threading, and performance metrics measured during initialization and repeated retrieval.

Each implementation was tested on a Windows 10 machine and connects to a cloud-hosted MongoDB Atlas database using the `pymongo` library. The connection URI was stored securely using a `.env` file loaded via `python-dotenv`.

2 Eager Initialization

2.1 Overview

Eager Initialization creates the Singleton instance at class load time, before any thread is spawned. Because Python's module import system is inherently sequential, the instance is guaranteed to exist by the time any method or thread accesses it, providing thread safety without any explicit locking mechanism.

2.2 Test Output

```
PS C:\Users\hp\Desktop\SDP_LAB> python -u "c:\Users\hp\Desktop\SDP_LAB\Eager_Initialization.py"

=====
Instance Validation: Eager
IDs: [1449487359952, 1449487359952, 1449487359952, 1449487359952, 1449487359952]
All same? True

=====
Thread Safety: Eager
Unique IDs collected: {1449487359952}
Thread-safe? True

=====
Performance: Eager
Init time: 0.000002s
Access time: 0.019881s
```

Figure 1: Eager Initialization

2.3 Results

- **Instance Validation:** All five calls returned the same object ID (1449487359952), confirming a single instance is enforced. (**PASSED**)
- **Thread Safety:** Only one unique ID was collected across all concurrent threads ({1449487359952}), confirming the implementation is **SAFE**.
- **Performance:** Init time was 0.000002s; Access time was 0.019881s. Near-zero init overhead because the instance is created at class definition time, and retrieval is a direct class-variable lookup.

3 Lazy Initialization

3.1 Overview

Lazy Initialization defers instance creation until the first call to `get_instance()`. This avoids upfront resource allocation but introduces a race condition in multi-threaded environments, as multiple threads can simultaneously pass the `if _instance is None` check before any one of them completes construction.

3.2 Test Output

```

PS C:\Users\hp\Desktop\SDP_LAB> python -u "c:\Users\hp\Desktop\SDP_LAB\Lazy_Initialization.py"

=====
Instance Validation: Lazy
IDs: [2059632631760, 2059632631760, 2059632631760, 2059632631760, 2059632631760]
All same? True
PASSED

=====
Thread Safety: Lazy
Unique IDs collected: {2059644636288, 2059643547136, 2059644441120, 2059644249168, 2059633051920, 2059633053520, 2059643615056, 2059643547408, 2059644438992, 2059643614800}
Thread-safe? False

=====
Performance: Lazy
Init time: 0.000002s
Access time: 0.020688s

```

Figure 2: Lazy Initialization

3.3 Results

- **Instance Validation:** All five single-threaded calls returned the same object ID (2059632631760), confirming correct behavior in a non-concurrent context. (**PASSED**)
- **Thread Safety:** Ten distinct IDs were collected, confirming a new instance was created by each thread. This implementation is **UNSAFE** under concurrent access.
- **Performance:** Init time was 0.000002s; Access time was 0.020688s. Fast access due to a single `None` check, but safety is not guaranteed.

4 Synchronized Method

4.1 Overview

The Synchronized Method wraps the entire `get_instance()` body inside a `threading.Lock()`. This guarantees that only one thread can execute the method at a time, making it thread-safe. The cost is that the lock is acquired on *every* call, even after the instance already exists.

4.2 Test Output

```
PS C:\Users\hp\Desktop\SDP_LAB> python -u "c:\Users\hp\Desktop\SDP_LAB\Synchronized_method.py"

=====
Instance Validation: Synchronized
IDs: [2486215434192, 2486215434192, 2486215434192, 2486215434192, 2486215434192]
All same? True

=====
Thread Safety: Synchronized
Unique IDs collected: {2486215854672}
Thread-safe? True

=====
Performance: Synchronized
Init time: 0.000004s
Access time: 0.038511s
```

Figure 3: Synchronized Method

4.3 Results

- **Instance Validation:** All five calls returned the same object ID (2486215434192). (**PASSED**)
- **Thread Safety:** Only one unique ID was collected ({2486215434192}), confirming the implementation is **SAFE**.
- **Performance:** Init time was 0.000004s; Access time was 0.038511s. The highest access overhead of all implementations due to lock acquisition on every retrieval call.

5 Double-Checked Locking

5.1 Overview

Double-Checked Locking optimizes the Synchronized Method by adding an outer `if _instance is None` check before acquiring the lock. Once the instance exists, subsequent calls bypass the lock entirely, achieving thread safety with near-Lazy access speed on all calls after the first.

5.2 Test Output

```
PS C:\Users\hp\Desktop\SDP_LAB> python -u "c:\Users\hp\Desktop\SDP_LAB\Double_Checked_Locking.py"

=====
Instance Validation: Double-Checked
IDs: [2223035310032, 2223035310032, 2223035310032, 2223035310032, 2223035310032]
All same? True

=====
Thread Safety: Double-Checked
Unique IDs collected: {2223035310032}
Thread-safe? True

=====
Performance: Double-Checked
Init time: 0.000002s
Access time: 0.014190s
```

Figure 4: Double-Checked Locking – Instance Validation, Thread Safety, and Performance

5.3 Results

- **Instance Validation:** All five calls returned the same object ID (2223035310032). (**PASSED**)
- **Thread Safety:** Only one unique ID was collected ({2223035310032}), confirming the implementation is **SAFE**.
- **Performance:** Init time was 0.000002s; Access time was 0.014190s. Significantly faster than Synchronized due to the outer null-check bypassing the lock after initialization.

6 Bill Pugh Implementation

6.1 Overview

The Bill Pugh approach implements the Singleton via Python's `__new__` method combined with double-checked locking. It delegates instance control to the object construction layer rather than a static method, making it arguably the most Pythonic approach while retaining explicit lock-based thread safety.

6.2 Test Output

```
PS C:\Users\hp\Desktop\SDP_LAB> python -u "c:\Users\hp\Desktop\SDP_LAB\Bill_Pugh_Implementation.py"

=====
Instance Validation: Bill Pugh
IDs: [1536611508176, 1536611508176, 1536611508176, 1536611508176, 1536611508176]
All same? True

=====
Thread Safety: Bill Pugh
Unique IDs collected: {1536611508176}
Thread-safe? True

=====
Performance: Bill Pugh
Init time: 0.000002s
Access time: 0.017180s
```

Figure 5: Bill Pugh Implementation – Instance Validation, Thread Safety, and Performance

6.3 Results

- **Instance Validation:** All five calls returned the same object ID (1536611508176). (**PASSED**)
- **Thread Safety:** Only one unique ID was collected ({1536611508176}), confirming the implementation is **SAFE**.
- **Performance:** Init time was 0.000002s; Access time was 0.017180s. Slightly higher than Double-Checked due to `__new__` invocation overhead on every call.

7 Enum-Based Singleton

7.1 Overview

The Enum-Based Singleton leverages Python's **Enum** metaclass, which guarantees that each named member is created exactly once and is immutable. No explicit locking is required. This approach is the most tamper-proof, as it cannot be broken by subclassing or reflection.

7.2 Test Output

```

PS C:\Users\hp\Desktop\SDP_LAB> python -u "c:\Users\hp\Desktop\SDP_LAB\Enum_Based_Singleton.py"

=====
Instance Validation: Enum
IDs: [1731142276432, 1731142276432, 1731142276432, 1731142276432, 1731142276432]
All same? True

=====

Thread Safety: Enum
Unique IDs collected: {1731142276432}
Thread-safe? True

=====

Performance: Enum
Init time: 0.000001s
Access time: 0.006779s

```

Figure 6: Enum-Based Singleton – Instance Validation, Thread Safety, and Performance

7.3 Results

- **Instance Validation:** All five calls returned the same object ID (1731142276432). (**PASSED**)
- **Thread Safety:** Only one unique ID was collected ({1731142276432}), confirming the implementation is **SAFE**.
- **Performance:** Init time was 0.000001s; Access time was 0.006779s. Fastest 1000-access time among all thread-safe implementations, benefiting from Python's highly optimized Enum attribute lookup.

8 Comparative Analysis

8.1 Instance Validation Comparison

Table 1: Memory Reference Validation Across All Implementations

Implementation	All IDs Identical?	Result
Eager Initialization	Yes (ID: 1449487359952)	PASSED
Lazy Initialization	Yes (ID: 2059632631760)	PASSED
Synchronized Method	Yes (ID: 2486215434192)	PASSED
Double-Checked Locking	Yes (ID: 2223035310032)	PASSED
Bill Pugh	Yes (ID: 1536611508176)	PASSED
Enum-Based	Yes (ID: 1731142276432)	PASSED

Observation: All six implementations successfully returned the same object reference across multiple instantiation calls in a single-threaded context, confirming that each correctly enforces the Singleton constraint.

8.2 Thread Safety Comparison

Table 2: Concurrent Access Consistency (10 Threads with Forced Race Condition)

Implementation	Unique Instances	Status	Mechanism
Eager Initialization	1	SAFE	Pre-instantiated at class load time
Lazy Initialization	10	UNSAFE	No guard; race condition exploited
Synchronized Method	1	SAFE	Full method lock on every call
Double-Checked Locking	1	SAFE	Outer null-check bypasses lock after init
Bill Pugh	1	SAFE	<code>__new__</code> with double-checked lock
Enum-Based	1	SAFE	Native Enum metaclass guarantee

Observation: Lazy Initialization was the only implementation to fail the thread safety test, producing 10 distinct instances under forced concurrency. All other implementations produced exactly one unique instance across all threads.

8.3 Performance Comparison

Table 3: Empirical Performance Comparison

Implementation	Init Time (s)	Access Time (s)
Eager Initialization	0.000002	0.019881
Lazy Initialization	0.000002	0.020688
Synchronized Method	0.000004	0.038511
Double-Checked Locking	0.000002	0.014190
Bill Pugh	0.000002	0.017180
Enum-Based Singleton	0.000001	0.006779

Observations:

- **Fastest initialization:** Enum Locking (0.000001s), followed closely by others (0.000002s).
- **Fastest Access:** Enum-Based (0.006779s), benefiting from Python's optimized Enum member lookup.

- **Slowest access:** Synchronized Method (0.038511s), almost $3\times$ slower than Double-Checked, due to lock acquisition on every call.
- **Slowest initialization:** Synchronized Method (0.000004s), reflecting the overhead of acquiring a lock even on first creation.

9 Summary

Table 4: Final Comparison of All Singleton Implementations

Implementation	Thread-Safe	Init Speed	Access Speed	Best Use Case
Eager	SAFE	Fastest	Fast	App-wide shared resources
Lazy	UNSAFE	Fast	Fast	Single-threaded only
Synchronized	SAFE	Slow	Slowest	Simple but low-performance
Double-Checked	SAFE	Fastest	Fast	Concurrent, performance-sensitive
Bill Pugh	SAFE	Fast	Moderate	Pythonic OOP approach
Enum-Based	SAFE	Fast	Fastest	Immutable, foolproof singleton

9.1 Key Findings

1. **Only unsafe implementation:** Lazy Initialization was the only pattern that failed thread safety, producing 10 distinct instances under forced concurrency.
2. **Best access performance among thread-safe options:** Enum-Based achieved the lowest 1000-access time (0.006779s), edging out Double-Checked (0.014190s).
3. **Worst access performance:** Synchronized Method (0.038511s) acquires a lock on every retrieval, making it almost $3\times$ slower than Double-Checked Locking.
4. **Best balance of safety and speed:** Double-Checked Locking and Enum-Based both achieve thread safety with fast post-init access by avoiding unnecessary locking.
5. **Most tamper-proof:** Enum-Based Singleton cannot be broken by subclassing, reflection, or concurrency due to Python's native Enum metaclass guarantees, making it the most robust option despite requiring attribute-style access.